

Python Portfolio — Database Schema

Project: Python Portfolio
Technology: Python, FastAPI, SQLAlchemy, SQLite, Pydantic, JWT
Location: /home/dominic/Documents/python-portfolio
Report #: 4/20

Database Engine: SQLite

Shared User Model

```
class User(Base):
    id: int (PK)
    email: str (unique)
    username: str (unique)
    hashed_password: str
```

Plus module-specific tables for shortener, chat, queue, and RAG modules. Schema auto-created via `Base.metadata.create_all()` on startup.

Database Schema

Entity Relationship Diagram



Tables

Table: User

Column	Type	Constraints	Description
id	Integer	PK, auto-increment	Primary key
email	String(255)	UNIQUE, NOT NULL	User email
username	String(100)	UNIQUE, NOT NULL	Display name
hashed_password	String(255)	NOT NULL	bcrypt hash
created_at	DateTime	NOT NULL, default=now	Account creation
is_active	Boolean	default=True	Account status

Table: Module-specific

Column	Type	Constraints	Description
id	Integer	PK	Primary key
(varies by module)	-	-	-

Table: Module-specific

Column	Type	Constraints	Description
id	Integer	PK	Primary key
(varies by module)	-	-	-

Relationships

- User has many resources (1:N)
- Resources belong to User (N:1)
- Module-specific entities linked via foreign keys

Indexes

Table	Index	Columns	Type	Purpose
users	ix_user_email	email	UNIQUE	Fast login lookup
urls	ix_url_short_code	short_code	UNIQUE	Fast redirect
tasks	ix_task_user	user_id	NON-UNIQUE	User task listing

Migration Strategy

- Currently using SQLite with auto-create via `Base.metadata.create_all()`
- Schema changes require manual migration script
- Recommended: Alembic for version-controlled migrations
- For production: migrate to PostgreSQL with Alembic

Performance Notes

- All foreign keys should be indexed
- Text search uses LIKE queries (consider FTS5 for SQLite)
- Large text fields (content, description) may benefit from compression